

Fiche de synthèse structurée sur les réseaux de neurones convolutionnels (CNN)

Introduction, convolutions, padding, stride, pooling, canaux multiples et LeNet

Cette fiche reprend et fusionne les contenus essentiels des documents fournis en une synthèse pédagogique, claire et progressive. Elle contient des définitions, les concepts clés, les formules indispensables, des explications détaillées, des exemples simples et des scripts Python commentés.

Version LaTeX compilable

8 avril 2026

Table des matières

1	Vue d'ensemble : pourquoi les CNN ?	2
1.1	Pourquoi un MLP est peu adapté aux images ?	2
1.2	Les trois idées fondatrices des CNN	2
2	Corrélation croisée 2D et convolution	2
2.1	Définition	2
2.2	Interprétation intuitive	3
2.3	Exemple numérique simple	3
2.4	Script Python commenté	3
3	Détection de contours	3
3.1	Filtre de contour horizontal local	4
3.2	Exemple Python commenté	4
4	Apprentissage automatique d'un noyau	4
4.1	Fonction de coût	4
4.2	Script Python commenté	4
5	Padding et stride	5
5.1	Formule générale	5
5.2	Cas classique : conserver la taille	5
5.3	Exemples simples	5
5.4	Script Python commenté	5
6	Pooling dans les CNN	6
6.1	Deux types classiques	6
6.1.1	Max-pooling	6
6.1.2	Average-pooling	6
6.2	Exemple simple	6
6.3	Taille de sortie	7
6.4	Script Python commenté	7
6.5	Version PyTorch	7
7	Canaux multiples dans les convolutions	8
7.1	Définitions	8
7.2	Formule	8
7.3	Exemple simple	8
7.4	Script Python commenté	8
8	Convolution 1×1	9
8.1	Formulation	9
8.2	Script Python commenté	9
9	Architecture LeNet	9
9.1	Architecture étudiée	9
9.2	Détail des couches	10
9.3	Formule de taille de sortie en convolution	10
9.4	Application à LeNet	10
9.5	Script Python commenté	10
9.6	Inspection des dimensions	11

10 Comparaison synthétique : MLP vs CNN	11
11 Erreurs fréquentes à éviter	11
12 Résumé final	12

1 Vue d'ensemble : pourquoi les CNN ?

Les images possèdent une **structure spatiale**. Deux pixels voisins sont souvent fortement liés, alors que deux pixels éloignés le sont beaucoup moins. Une couche entièrement connectée (MLP) traite pourtant l'image comme un simple vecteur plat, ce qui détruit l'organisation 2D naturelle des données.

Définition

Un **réseau de neurones convolutionnel** (CNN) est un réseau qui applique des filtres locaux partagés sur une image ou sur des cartes de caractéristiques afin d'extraire automatiquement des motifs pertinents comme des bords, textures, formes ou structures plus complexes.

1.1 Pourquoi un MLP est peu adapté aux images ?

Si une image contient 10^6 pixels et qu'on relie cette image à une couche cachée de 10^3 neurones, alors une seule couche dense nécessite :

$$10^6 \times 10^3 = 10^9$$

paramètres, sans compter les biais.

Concepts clés

- Un MLP ignore la géométrie de l'image.
- Il exige énormément de mémoire et de données.
- Il ne réutilise pas les mêmes détecteurs à plusieurs endroits de l'image.

1.2 Les trois idées fondatrices des CNN

1. **Localité** : une sortie dépend d'un petit voisinage local.
2. **Partage des poids** : le même filtre est utilisé partout.
3. **Hierarchie des représentations** : les couches profondes combinent progressivement les motifs simples pour former des motifs complexes.

2 Corrélation croisée 2D et convolution

En deep learning, le mot « convolution » désigne souvent en pratique une **corrélation croisée**. La différence mathématique stricte est que, dans la convolution classique, le noyau est retourné. En apprentissage profond, cette nuance est peu gênante puisque les noyaux sont appris.

2.1 Définition

Soient :

- $X \in \mathbb{R}^{H \times W}$ l'entrée ;
- $K \in \mathbb{R}^{h \times w}$ le noyau.

La corrélation croisée 2D est définie par :

$$Y(i, j) = \sum_{a=0}^{h-1} \sum_{b=0}^{w-1} X(i+a, j+b) K(a, b).$$

La taille de sortie, sans padding et avec stride égal à 1, est :

$$(H - h + 1) \times (W - w + 1).$$

2.2 Interprétation intuitive

Le noyau glisse sur l'image. À chaque position, on extrait une petite fenêtre, on effectue un produit terme à terme avec le noyau, puis on additionne le tout. La valeur produite mesure donc la présence locale du motif représenté par le filtre.

2.3 Exemple numérique simple

Soient

$$X = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}, \quad K = \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}.$$

Alors le premier élément de sortie vaut :

$$0 \cdot 0 + 1 \cdot 1 + 3 \cdot 2 + 4 \cdot 3 = 19.$$

2.4 Script Python commenté

```

1 import torch
2
3 def corr2d(X, K):
4     """
5     X : matrice 2D d'entree
6     K : noyau 2D
7     Retourne la corrélation croisée 2D entre X et K
8     """
9     h, w = K.shape
10
11     # Taille de sortie compatible avec le noyau
12     Y = torch.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
13
14     for i in range(Y.shape[0]):
15         for j in range(Y.shape[1]):
16             # Extraction de la fenetre locale
17             region = X[i:i+h, j:j+w]
18             # Produit terme a terme puis somme
19             Y[i, j] = (region * K).sum()
20
21     return Y

```

3 Détection de contours

Une convolution peut être vue comme un détecteur de motif local. Par exemple, un filtre très simple peut détecter un changement brutal d'intensité.

3.1 Filtre de contour horizontal local

Le noyau

$$K = \begin{bmatrix} 1 & -1 \end{bmatrix}$$

calcule essentiellement la différence entre deux pixels voisins selon l'axe horizontal. Il détecte donc des **contours verticaux**.

Interprétation du signe

- sortie positive : passage d'une région claire vers une région sombre ;
- sortie négative : passage d'une région sombre vers une région claire ;
- sortie nulle : peu ou pas de variation locale.

3.2 Exemple Python commenté

```

1 import torch
2
3 # Image artificielle : fond blanc avec une bande noire au milieu
4 X = torch.ones((6, 8))
5 X[:, 2:6] = 0
6
7 # Filtre simple de detection de contraste horizontal
8 K = torch.tensor([[1.0, -1.0]])
9
10 Y = corr2d(X, K)
11 print(Y)

```

4 Apprentissage automatique d'un noyau

Il n'est pas nécessaire de choisir manuellement les filtres. Une couche convolutionnelle peut apprendre automatiquement un noyau à partir de données d'entrée et de sortie cible.

4.1 Fonction de coût

Une perte quadratique classique s'écrit :

$$L = \sum_{i,j} (Y_{\text{pred}}(i,j) - Y(i,j))^2.$$

La mise à jour des paramètres suit alors la descente de gradient :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L.$$

4.2 Script Python commenté

```

1 import torch
2 from torch import nn
3
4 conv2d = nn.LazyConv2d(1, kernel_size=(1, 2), bias=False)
5

```

```

6 # PyTorch attend la forme (batch, canaux, hauteur, largeur)
7 X_train = X.reshape((1, 1, 6, 8))
8 Y_target = Y.reshape((1, 1, 6, 7))
9
10 lr = 3e-2
11
12 for epoch in range(10):
13     Y_hat = conv2d(X_train)
14     loss = (Y_hat - Y_target) ** 2
15
16     conv2d.zero_grad()
17     loss.sum().backward()
18
19     # Mise à jour manuelle des poids
20     conv2d.weight.data[:] -= lr * conv2d.weight.grad
21
22     if (epoch + 1) % 2 == 0:
23         print(f"epoch {epoch + 1}, loss {loss.sum():.3f}")

```

5 Padding et stride

Le **padding** ajoute des bordures artificielles, souvent remplies de zéros, autour de l'image. Le **stride** contrôle de combien de cases le noyau se déplace à chaque étape.

5.1 Formule générale

Pour une entrée de taille $n_h \times n_w$, un noyau $k_h \times k_w$, un padding (p_h, p_w) et un stride (s_h, s_w) :

$$H_{out} = \left\lfloor \frac{n_h - k_h + 2p_h}{s_h} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{n_w - k_w + 2p_w}{s_w} \right\rfloor + 1.$$

5.2 Cas classique : conserver la taille

Pour un noyau impair, on choisit souvent :

$$p_h = \frac{k_h - 1}{2}, \quad p_w = \frac{k_w - 1}{2}.$$

Par exemple, avec un noyau 3×3 , on prend généralement $p = 1$ pour conserver la taille spatiale.

5.3 Exemples simples

- Entrée 8×8 , noyau 3×3 , padding 1, stride 1 : sortie 8×8 .
- Entrée 8×8 , noyau 3×3 , padding 1, stride 2 : sortie 4×4 .
- Entrée 8×8 , noyau 5×3 , padding (2,1), stride (1,1) : sortie 8×8 .

5.4 Script Python commenté

```

1 import torch
2 from torch import nn
3
4 def comp_conv2d(conv2d, X):

```

```

5     # Ajout des dimensions batch et canal
6     X = X.reshape((1, 1) + X.shape)
7
8     # Application de la convolution
9     Y = conv2d(X)
10
11    # Suppression des dimensions batch et canal pour une lecture simple
12    return Y.reshape(Y.shape[2:])
13
14 X = torch.rand(size=(8, 8))
15
16 # Exemple 1 : conserver la taille
17 conv_same = nn.LazyConv2d(1, kernel_size=3, padding=1)
18 print(comp_conv2d(conv_same, X).shape) # attendu : (8, 8)
19
20 # Exemple 2 : reduire la resolution avec stride = 2
21 conv_stride = nn.LazyConv2d(1, kernel_size=3, padding=1, stride=2)
22 print(comp_conv2d(conv_stride, X).shape) # attendu : (4, 4)

```

6 Pooling dans les CNN

Le **pooling** résume localement l'information contenue dans une carte de caractéristiques. Contrairement à la convolution, il ne possède pas de poids à apprendre.

Définition

Une couche de pooling parcourt l'entrée avec une fenêtre locale et remplace chaque région par une seule valeur, souvent le maximum ou la moyenne.

6.1 Deux types classiques

6.1.1 Max-pooling

$$Y_{i,j} = \max_{0 \leq a < p_h, 0 \leq b < p_w} X_{i+a,j+b}.$$

Il conserve l'activation la plus forte de la région.

6.1.2 Average-pooling

$$Y_{i,j} = \frac{1}{p_h p_w} \sum_{a=0}^{p_h-1} \sum_{b=0}^{p_w-1} X_{i+a,j+b}.$$

Il calcule une moyenne locale plus lissée.

6.2 Exemple simple

Pour

$$X = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix},$$

avec une fenêtre 2×2 :

$$Y_{\max} = \begin{bmatrix} 4 & 5 \\ 7 & 8 \end{bmatrix}, \quad Y_{\text{avg}} = \begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix}.$$

6.3 Taille de sortie

Comme pour la convolution :

$$H_{\text{out}} = \left\lfloor \frac{H - K_h + 2P_h}{S_h} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W - K_w + 2P_w}{S_w} \right\rfloor + 1.$$

À retenir

Le pooling traite chaque canal **indépendamment**. Le nombre de canaux est donc conservé.

6.4 Script Python commenté

```

1 import torch
2
3 def pool2d(X, pool_size, mode='max'):
4     """
5     X : matrice 2D
6     pool_size : (hauteur, largeur)
7     mode : 'max' ou 'avg'
8     """
9     p_h, p_w = pool_size
10    Y = torch.zeros((X.shape[0] - p_h + 1, X.shape[1] - p_w + 1))
11
12    for i in range(Y.shape[0]):
13        for j in range(Y.shape[1]):
14            region = X[i:i+p_h, j:j+p_w]
15            if mode == 'max':
16                Y[i, j] = region.max()
17            elif mode == 'avg':
18                Y[i, j] = region.mean()
19            else:
20                raise ValueError("mode doit etre 'max' ou 'avg'")
21
22    return Y

```

6.5 Version PyTorch

```

1 import torch
2 from torch import nn
3
4 # Entree : (batch=1, channels=1, height=4, width=4)
5 X = torch.arange(16, dtype=torch.float32).reshape((1, 1, 4, 4))
6
7 # Max-pooling fenetre 3x3
8 pool = nn.MaxPool2d(3)
9 print(pool(X))
10
11 # Max-pooling fenetre 3x3, padding=1, stride=2

```

```

12 pool = nn.MaxPool2d(3, padding=1, stride=2)
13 print(pool(X))

```

7 Canaux multiples dans les convolutions

En pratique, une image réelle contient souvent plusieurs canaux. Une image RGB possède 3 canaux : rouge, vert et bleu. Les cartes internes d'un CNN ont elles aussi plusieurs canaux.

7.1 Définitions

Une entrée multi-canaux est un tenseur :

$$X \in \mathbb{R}^{c_i \times h \times w}$$

où c_i est le nombre de canaux d'entrée.

Pour produire c_o canaux de sortie, le noyau global a la forme :

$$K \in \mathbb{R}^{c_o \times c_i \times k_h \times k_w}.$$

7.2 Formule

Pour chaque canal de sortie j :

$$Y_j = \sum_{c=1}^{c_i} X_c \star K_{j,c}.$$

Chaque canal de sortie dépend donc de **tous** les canaux d'entrée.

7.3 Exemple simple

Si

$$X \in \mathbb{R}^{2 \times 3 \times 3}, \quad K \in \mathbb{R}^{3 \times 2 \times 2 \times 2},$$

alors la sortie a 3 canaux et une taille spatiale 2×2 :

$$Y \in \mathbb{R}^{3 \times 2 \times 2}.$$

7.4 Script Python commenté

```

1 import torch
2
3 def corr2d_multi_in(X, K):
4     # X : (c_i, h, w)
5     # K : (c_o, c_i, k_h, k_w)
6     # On somme les correlations sur tous les canaux
7     return sum(corr2d(x, k) for x, k in zip(X, K))
8
9 def corr2d_multi_in_out(X, K):
10    # X : (c_i, h, w)
11    # K : (c_o, c_i, k_h, k_w)
12    # On calcule un canal de sortie pour chaque noyau de sortie
13    return torch.stack([corr2d_multi_in(X, k) for k in K], dim=0)

```

8 Convolution 1×1

Une convolution 1×1 n'agrége pas les voisins spatiaux. Elle agit uniquement sur les canaux au même emplacement.

8.1 Formulation

Si

$$X \in \mathbb{R}^{c_i \times h \times w}, \quad K \in \mathbb{R}^{c_o \times c_i \times 1 \times 1},$$

alors à chaque position (u, v) :

$$Y(:, u, v) = W X(:, u, v), \quad W \in \mathbb{R}^{c_o \times c_i}.$$

Pourquoi c'est utile ?

- réduire ou augmenter le nombre de canaux ;
- mélanger plusieurs cartes de caractéristiques ;
- ajouter une transformation linéaire légère entre deux convolutions plus larges ;
- réduire le coût de calcul dans certains réseaux profonds.

8.2 Script Python commenté

```

1 import torch
2
3 def corr2d_multi_in_out_1x1(X, K):
4     c_i, h, w = X.shape
5     c_o = K.shape[0]
6
7     # On aplatit l'image : chaque colonne represente un pixel
8     X = X.reshape((c_i, h * w))
9
10    # Le noyau 1x1 devient une matrice de transformation
11    K = K.reshape((c_o, c_i))
12
13    # Multiplication matricielle
14    Y = torch.matmul(K, X)
15
16    # Remise en forme spatiale
17    return Y.reshape((c_o, h, w))

```

9 Architecture LeNet

LeNet-5 est l'une des premières architectures historiques de CNN ayant marqué la vision par ordinateur. Elle montre qu'il est préférable de conserver la structure spatiale des images plutôt que de les aplatir immédiatement.

9.1 Architecture étudiée

Chaîne des dimensions :

$$(1, 28, 28) \rightarrow (6, 28, 28) \rightarrow (6, 14, 14) \rightarrow (16, 10, 10) \rightarrow (16, 5, 5) \rightarrow 400 \rightarrow 120 \rightarrow 84 \rightarrow 10.$$

9.2 Détail des couches

Couche	Description
Entrée	image en niveaux de gris de taille $1 \times 28 \times 28$
Conv1	6 filtres, noyau 5×5 , padding 2
Activation	sigmoïde
Pooling 1	average pooling 2×2 , stride 2
Conv2	16 filtres, noyau 5×5 , sans padding
Activation	sigmoïde
Pooling 2	average pooling 2×2 , stride 2
Flatten	vectorisation
FC1	120 neurones
FC2	84 neurones
Sortie	10 neurones (classes)

9.3 Formule de taille de sortie en convolution

$$H_{out} = \left\lfloor \frac{H - K_h + 2P_h}{S_h} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W - K_w + 2P_w}{S_w} \right\rfloor + 1.$$

9.4 Application à LeNet

- après Conv1 : $6 \times 28 \times 28$;
- après AvgPool1 : $6 \times 14 \times 14$;
- après Conv2 : $16 \times 10 \times 10$;
- après AvgPool2 : $16 \times 5 \times 5$;
- après Flatten : $16 \times 5 \times 5 = 400$.

9.5 Script Python commenté

```

1 import torch
2 from torch import nn
3
4 class LeNet(nn.Module):
5     def __init__(self, num_classes=10):
6         super().__init__()
7         self.net = nn.Sequential(
8             nn.LazyConv2d(6, kernel_size=5, padding=2),
9             nn.Sigmoid(),
10            nn.AvgPool2d(kernel_size=2, stride=2),
11
12            nn.LazyConv2d(16, kernel_size=5),
13            nn.Sigmoid(),
14            nn.AvgPool2d(kernel_size=2, stride=2),
15
16            nn.Flatten(),
17            nn.LazyLinear(120),
18            nn.Sigmoid(),
19            nn.LazyLinear(84),
20            nn.Sigmoid(),
21            nn.LazyLinear(num_classes)
22        )

```

```

23
24     def forward(self, x):
25         return self.net(x)

```

9.6 Inspection des dimensions

```

1  import torch
2
3  model = LeNet()
4  X = torch.randn(1, 1, 28, 28)
5
6  for layer in model.net:
7      X = layer(X)
8      print(layer.__class__.__name__, X.shape)

```

Ce script est très utile pour vérifier la cohérence de l'architecture et comprendre l'évolution progressive des dimensions.

10 Comparaison synthétique : MLP vs CNN

MLP	CNN
Aplatissement immédiat de l'image	Conservation de la structure 2D
Très grand nombre de paramètres	Partage des poids
Pas de notion locale explicite	Détection locale par convolution
Peu adapté aux images	Très adapté aux images

11 Erreurs fréquentes à éviter

1. **Confondre convolution et corrélation croisée** : vérifier si le noyau est retourné ou non.
2. **Oublier la forme de PyTorch** : les tenseurs sont souvent de la forme (N, C, H, W) .
3. **Mal calculer la taille de sortie** : toujours appliquer correctement la formule avec le plancher $\lfloor \cdot \rfloor$.
4. **Croire qu'une convolution 1×1 regarde les voisins** : elle mélange seulement les canaux au même pixel.
5. **Croire que le pooling fusionne les canaux** : faux, il traite chaque canal séparément.
6. **Oublier que le nombre de canaux de sortie correspond au nombre de filtres.**

12 Résumé final

À retenir absolument

- Les images possèdent une structure spatiale qu'un MLP ne respecte pas naturellement.
- Les CNN exploitent deux idées majeures : la **localité** et le **partage des poids**.
- La corrélation croisée 2D est l'opération de base des couches convolutionnelles.
- Le **padding** contrôle la taille de sortie et limite la perte d'information sur les bords.
- Le **stride** permet de sous-échantillonner et de réduire le coût de calcul.
- Le **pooling** résume localement les activations sans paramètres appris.
- Les **canaux multiples** enrichissent la représentation et les convolutions 1×1 permettent de les mélanger efficacement.
- **LeNet** est une architecture fondatrice qui illustre clairement l'enchaînement convolution + activation + pooling + couches denses.